

웹 공격 페이로드 이미지화 기반 캐스케이드 탐지 및 적대적 강건성 연구

제안 모델을 단일 CNN → 캐스케이드로 전환한 이후의 전 과정과 실측을 논문 서술 순서 그대로 한 흐름으로 정리

작성일 2026-08-12 · 범위 Phase 10 ~ 12 전 구간(2026-07-25 ~ 2026-08-12, 캐스케이드 전환 이후 전부) · 근거 문서 docs/09 · docs/10 · docs/11 · docs/12 및 마스터 설계 문서

이 문서가 하는 일

캐스케이드로 주제를 전환한 뒤 진행된 모든 설계 결정·실측·부정 결과를 논문 목차에 바로 엮을 수 있는 하나의 흐름으로 묶었습니다. 각 절은 논문의 어느 장으로 가는지(\$12 매핑표), 어떤 문구로 써야 하고 어떤 문구를 쓰면 안 되는지(\$10 기여표)까지 함께 적었습니다. 수치는 전부 기존 설계 문서·산출 JSON 에 근거하며 이 문서에서 새로 만든 수치는 없습니다.

다섯 줄 요약 — 논문의 전체 이야기

- 출발은 음성 결과입니다. 이미지화 CNN 단독은 텍스트 모델에 유의하게 집니다(char-CNN MCC 0.9944 vs RGB CNN 0.9762, 5-fold CV + Holm 보정 adj $p = 0.0010 \sim 0.0247$). 대신 처리량이 13.6배 우위입니다. 즉 이기는 측은 정확도가 아니라 비용이었습니다.
- 그래서 제안 모델을 캐스케이드로 바꿨습니다. 1차 RGB CNN 이 전량을 보고, 확신도가 τ 미만인 소수만 2차 char-CNN 으로 올립니다. 트래픽의 8.0%만 올려 char-CNN 급 정확도를 유지하면서 지연을 5.07배 낮췄고, 추가 학습은 0입니다(기존 체크포인트 2개의 조합).
- 구조를 바꾸자 단일 모델에서는 존재하지 않던 축 두 개가 드러났습니다. 정확도가 아니라 지연을 표적으로 하는 비용 기반 공격(에스컬레이션 2.17배, SLA 붕괴)과, 방어의 대가가 정확도가 아니라 지연으로 지불된다는 현상(esc 34.97% → 69.52%)입니다.
- 방어 실험의 헤드라인은 학습형 방어의 패배입니다. 미지 변형 조건에서 학습 8셀짜리 적대적 증강을 학습이 0인 입력 정규화가 이깁니다(RGB CNN 0.2112 vs 0.4693/0.5704).
- Phase 12 확장 3안 중 논문의 주장을 키운 것은 하나뿐이며, 부정 결과 2건을 그대로 실습니다. 인코딩 불변 채널은 단독으로는 실패했지만 증강과 결합하면 미지 변형 일반화를 10.9% → 69.2% 로 올려 이 프로젝트에서 가장 아팠던 음성 결과(F9)를 부분적으로 뒤집었습니다.

0. 읽는 법 — 이 문서의 구조

절 순서는 마스터 설계 문서 §1.3 이 확정한 논문 서술 순서를 그대로 따릅니다. 번호가 뒤섞여 보이는 것은 RQ ID 를 재배열하지 않기로 못 박았기 때문입니다(ID 가 파일명에 박혀 있어 재번호는 12개 문서 136곳의 상호참조를 혼듭니다).

```
RQ1 (음성 결과: 이미지 단독은 진다) → §1
└─ RQ5 (그래서 캐스케이드 – 이 논문의 제안) → §2 · §3
    ├── RQ2 (회피에 얼마나 버티는가 + 비용 공격) → §4
    ├── RQ3 (방어를 붙이면 어떻게 되는가) → §5 · §6
    └─ RQ6 (제안 구조에서만 공짜로 얻어지는 능력) → §7
RQ4 (별도 축: 내용 기반 탐지 자체의 경계) → §8
```

그리고 §9(선행연구 대조) · §10(기여 문구와 금지어) · §11(한계 전체) · §12(논문 목차 매핑과 그림 인벤토리) · §13(재현 절차) 순으로 논문 작성에 바로 쓰는 재료를 붙였습니다.

Phase	한 일	결과 문서	판정
Phase 10	캐스케이드 탐지기 설계·구현·GPU 실측 (RQ5)	docs/09	채택 — 제안 모델
Phase 11	적대적 방어 학습 A(증강) · B(정규화) + 캐스케이드 편입 (RQ3)	docs/10	완료 — H3-4 는 학습형 방어의 패배
Phase 12	비용 축 선행연구 전수 대조 → 주장 강도 하향	docs/12	F7·M1 하향, M4 강등
Phase 12 / M1	표현 불일치 → 회피 시도 탐지 (RQ6)	docs/11 §13	⚠ 보조 지표로 하향
Phase 12 / M4	다단 조기종료로 1차 비용 절감	docs/11 §14	✗ 미채택(부정 결과)
Phase 12 / M2	인코딩 불변 채널 인코더	docs/11 §15	단독 미채택 / 증강 결합 시 채택

표 0. 이 문서가 다루는 범위 — 캐스케이드 전환 이후의 Phase

1. 출발점 — RQ1 의 답은 "아니오"였다 (논문 5장 도입)

1.1 실측이 만든 딜레마

원래 RQ1 은 "이미지 기반 CNN 이 텍스트 모델과 유사하거나 더 나은 성능을 보이는가"였고, 5-fold CV + paired t-test + Holm 보정으로 낸 답은 **아니오**였습니다.

모델	clean Macro-F1	MCC	ms/sample	처리량	Pareto 상 위치
gray CNN (48×48×1)	0.9676	0.9418	0.0066	92,466 /s	정확도 하위 · 비용 최저
RGB CNN (48×48×3)	0.9804	0.9725	0.0066	81,964 /s	RGB 로 격차를 좁혔으나 여전히 열세
char-CNN (텍스트)	0.9954	0.9940	0.0561	6,010 /s	정확도 최상위 · 13.6배 느림
BiLSTM (텍스트)	0.9838	0.9736	0.1468	—	가장 느림
(참고) 특징 융합형 — 두 브랜치 상시	—	—	0.0627	—	char-CNN 단독보다도 비쌈

표 1. 단일 모델 비교 (트랙 `payload_4class_csicnorm`, Normal = CSIC 2010 실트래픽 / 지연은 배치 512 순전파, 5라운드 × 20회 중앙값)

통계 검정으로 확인한 사실 3개(트랙 `payload_4class`, 5-fold CV):

- **F1** — 이미지화 CNN 은 텍스트 베이스라인에 **유의하게 열세**: char-CNN MCC 0.9944 ± 0.0004 vs RGB CNN 0.9762 ± 0.0025 , adj $p = 0.0010 \sim 0.0247$.
- **F2** — gray → RGB 개선은 **진짜**: MCC $0.9493 \rightarrow 0.9762 (+2.69pp)$, adj $p = 0.0037 \sim 0.0201$. (교수님이 요구하신 RGB 채널화가 실제로 효과가 있음을 통계로 확인한 부분입니다.)
- **F3** — 그러나 **채널 조합 간 우열은 판정 불가**: 6쌍 전부 adj $p \geq 0.88$. → "어떤 채널 조합이 낫다"는 서술은 논문에 쓰지 않습니다.

⚠ 정직성 원칙 (HARKing 방지) — 논문 서술에서 반드시 지킬 것

1. 원 RQ1 문구와 그 답이 "아니오"였음을 그대로 보고합니다. 숨기거나 바꿔치지 않습니다.
2. RQ5(캐스케이드)는 그 음성 결과에서 파생된 질문임을 명시합니다.
3. Phase 12 이후의 판정 기준(H5·H6·H7)은 실측 전에 문서로 고정했습니다(docs/11 §7).
이렇게 쓰면 "질문을 살짝 바꾼 것"이 아니라 "음성 결과에서 출발한 설계"가 되며, 선행 8편 중 유의성 검정을 한 논문이 0편인 상황에서 오히려 방법론적 강점이 됩니다.

1.2 결가지 두 개는 닫혔다 (논문에 부정 결과로 남김)

- **F13 — ViT / hybrid(CNN stem + Transformer) 미채택:** hybrid vs CNN 평균차 **+0.12pp**, adj $p = 1.0000$. 부수 발견으로, 기본 $lr = 1e-3$ 은 CNN 기준값이라 **단일 ViT 는 이 값에서 발산합니다** (lr 만 $3e-4/1e-4$ 로 낮추면 MCC 가 최대 +23.9pp 이동). "같은 학습 루프를 썼으니 공정하다"가 성립하지 않는 사례로 논문 실험환경 장에 씁니다.
- **F4 — clean 탐지 상보성 이득 없음:** 캐스케이드와 char-CNN 의 탐지 집합이 **완전히 동일**(둘 다 놓친 8건). → 캐스케이드의 기여는 탐지 집합 확장이 아니라 **비용 축소**임을 못 박는 근거입니다.

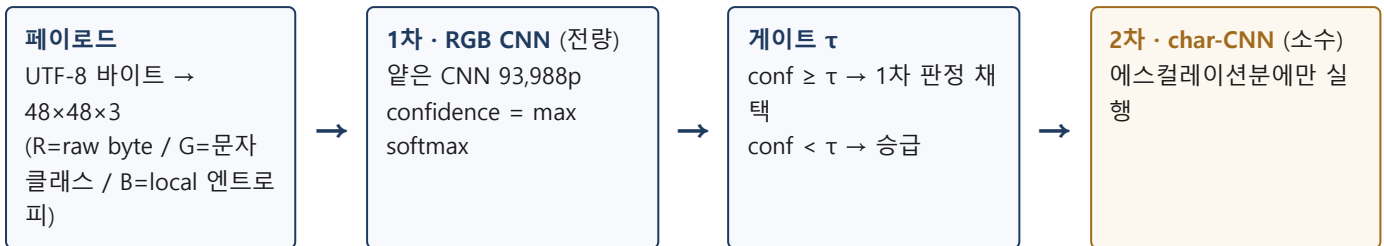
2. 제안 방법 — 왜 '특징 융합'이 아니라 '캐스케이드'인가 (논문 3장)

2.1 가장 중요한 설계 결정

"하이브리드"라고 하면 보통 two-branch **특징 융합**을 떠올립니다(이미지 CNN 특징 ⊕ char-CNN 특징 → concat → FC). 그러나 이 구조는 모든 입력이 두 브랜치를 **전부** 통과하므로 비용이

$$C_{\text{fusion}} = C_{\text{cnn}} + C_{\text{charcnn}} \quad (> \text{char-CNN 단독})$$

이 되어 **애초 목표였던 RGB CNN 의 속도 우위가 사라집니다**. 정확도만 얻고 속도는 잃으며, 심지어 char-CNN 단독보다 느립니다(실측 0.0627 ms vs 0.0561 ms). → **채택하지 않습니다**. 대신 **선택적 실행(cascade)** 을 택합니다.



$$C_{\text{cascade}} = C_{\text{cnn}} + r \cdot C_{\text{charcnn}} \quad (r = \text{에스컬레이션 비율})$$

r 이 작으면 **정확도는 char-CNN 급, 지연은 CNN 급**이 됩니다. τ 를 움직이면 정확도-지연 곡선이 그려지고, 양 끝 점은 정확히 두 단독 모델입니다 — $\tau \rightarrow 0$ 은 얇은 CNN 단독, $\tau > 1$ 은 char-CNN 단독. 이 성질은 **단위 테스트로 고정**했고 실측으로도 확인했습니다($\tau=0 \rightarrow \text{esc } 0.0000 / \text{F1 } 0.9676$, $\tau=1.01 \rightarrow \text{esc } 1.0000 / \text{F1 } 0.9954$).

학습은 하지 않습니다. Phase 4/5 에서 이미 학습한 체크포인트 2개를 그대로 조합합니다 → 추가 GPU 학습 비용 0, 그리고 "같은 모델을 배치 방식만 바꿨다"는 인과가 성립합니다.

2.2 공정성을 지키는 세 가지 못 (논문 4장 실험환경)

1. **τ 는 반드시 val 에서 고른다.** test 로 고르면 임계값이 test 에 과적합돼 성능이 부풀려집니다(정보 누수). val 스윕으로 τ 를 확인한 뒤 **test 에 한 번만** 적용합니다. 선택 규칙은 `match-teacher` (val Macro-F1 이 char-CNN 이 상이 되는 τ 중 에스컬레이션 최소)와 `budget` (에스컬레이션 ≤ 예산 아래 val Macro-F1 최대) 2종입니다.
2. **비교 기준선 4종을 같은 test-같은 지표로 함께 낸다** — 1차 단독 / 2차 단독 / 융합형 비용선 / **oracle** 라우팅(1차가 틀린 샘플만 정확히 넘겼을 때의 상한). oracle 은 **게이트 품질**을 재는 자로 씁니다.
3. **지연은 두 모델을 같은 기준으로 잰다.** warmup 후 **5라운드 × 20회의 중앙값**, CUDA 는 `synchronize()` 필수. 모델 순전파 기준이며 전처리는 제외(μs 수준이라 결론 불변).

⚠ 이 못이 실제로 필요했던 사례 — 논문 4장에 재현성 항목으로 씁니다. 1차 측정에서 char-CNN 지연이 같은 조건인데도 0.0555~0.0904 ms 로 튀어 특정 arm 의 speedup 이 **9.30배로 부풀려졌습니다**(재측정 7.05배). 원인은 단발 측정(warmup 3 · repeats 10)이 GPU 클럭 부스트 상태와 타 프로세스 간섭을 그대로 받은 것이었습니다. 5라운드 × 20회 중앙값으로 바꾼 뒤 변동은 0.7% 이내입니다. **헤드라인이 속도인 연구에서 지연 측정의 분산 관리는 정확도 지표만큼 중요합니다**.

3. RQ5 결과 — 같은 정확도를 몇 분의 1 비용으로 (논문 5장 본체)

3.1 arm 7종 전체 결과

1차	2차	τ 규칙	τ	esc	Macro-F1	MCC	ms/sample	spd	ovh
gray	char-CNN	match-teacher	0.9998	78.36%	0.9954	0.9940	0.0508	1.11×	7.74×
gray	char-CNN	budget 10%	0.7509	7.46%	0.9856	0.9756	0.0108	5.23×	1.64×
gray	char-CNN	budget 5%	0.5678	2.54%	0.9754	0.9579	0.0080	7.05×	1.22×
gray	BiLSTM	match-teacher	0.8099	9.98%	0.9831	0.9714	0.0212	6.92×	3.23×
RGB	char-CNN	match-teacher	0.9971	27.73%	0.9955	0.9941	0.0223	2.52×	3.32×
RGB	char-CNN	budget 10%	0.9425	7.99%	0.9934	0.9907	0.0111	5.07×	1.68×
RGB	char-CNN	budget 5%	0.7516	2.71%	0.9890	0.9837	0.0081	6.92×	1.23×

표 2. 캐스케이드 arm 7종 (RTX 4080 SUPER, val 22,628 / test 22,629, 클래스 = CommandInjection · Normal · SQLInjection · XSS). **esc** = 에스컬레이션 비율, **spd** = 2차 단독 대비 배속, **ovh** = 1차 단독 대비 비용.

헤드라인 (기여 C1) — RGB 1차 + 예산 10% 를 대표 운영점으로 삼습니다.

트래픽의 **8.0%만 2차로 올려** char-CNN 대비 MCC 0.9940 → 0.9907(−0.33pp)에 **지연은 5.07배 낮습니다**(0.0561 → 0.0111 ms/sample). 융합형(0.0627 ms) 대비로는 5.6배입니다. 정확도를 한 치도 양보하지 않으려면 **RGB + match-teacher** 가 27.7% 에스컬레이션으로 char-CNN 을 **넘어서면서**(F1 0.9955 vs 0.9954) 2.52배 빠릅니다.

⚠ 과장 금지 못 — 논문에서 정확도 개선을 주장하지 않습니다. 이 트랙의 clean 지표는 **표면 토큰 포화** 상태이고, 위 정확도 차이는 단일 split 노이즈($\pm 0.11\text{pp}$, cuDNN 비결정성)와 같은 자릿수입니다. 주장의 무게는 전부 **같은 정확도에서의 비용 절감폭**에 둡니다 — 이쪽은 노이즈에 훨씬 둔감합니다.

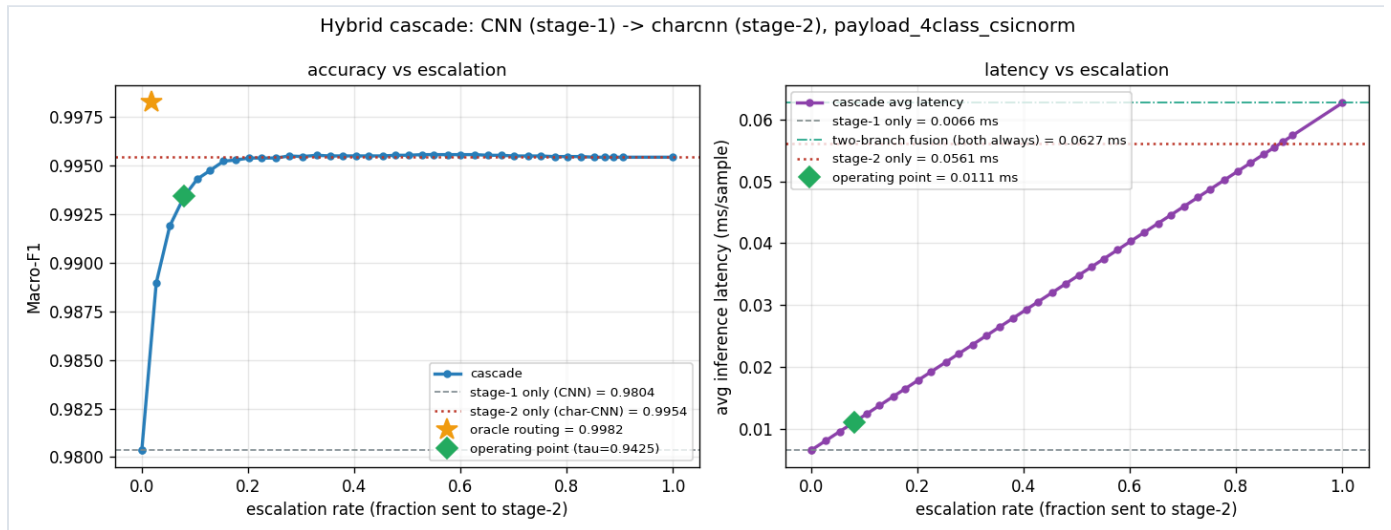


그림 1. 대표 운영점(RGB 1차 · budget 10%)의 정확도 / 지연 vs 에스컬레이션 곡선. τ 하나로 두 단독 모델 사이를 연속 이동하며, 양 끝점이 각각 같은 CNN 단독과 char-CNN 단독으로 수렴한다. (docs/figures/models/cascade_curve_..._rgb_b0.1_bal.png)

3.2 게이트 품질 — 이 Phase 의 가장 큰 미결 (기여 C4)

1차	oracle esc	oracle Macro-F1	실측 게이트가 같은 수준에 쓰는 esc	과잉 배수
gray	3.96%	0.9968	78.36%	19.8배
RGB	1.87%	0.9982	27.73%	14.8배

표 3. oracle 라우팅 대비 게이트 품질. oracle 은 "1차가 틀린 샘플만 정확히 넘겼을 때"의 상한이므로 어떤 확신도 게이트도 이보다 잘할 수 없다.

확신도(max softmax)는 오답을 **15~20배 과잉 포착**합니다. 문제는 정확도 상한이 아니라 **비용 상한**이며, 게이트를 개선하면(엔트로피 · 상위 2개 마진 · temperature scaling) 정확도를 그대로 두고 에스컬레이션만 최대 **-25.8pp** 줄일 여지가 있습니다. 논문에서는 이를 **"개선 여지의 상한을 못 박았다"**는 기여(C4)로 쓰고, 동시에 **후속 1순위 과제**로 명시합니다.

3.3 1차를 gray 로 쓸지 RGB 로 쓸지 — RGB 채택

정확도뿐 아니라 **비용에서도** RGB 가 이깁니다. char-CNN 급 정확도에 도달하는 데 gray 는 78.4% 를 넘겨야 했지만 RGB 는 27.7% 로 충분했습니다(1차가 정확할수록 넘길 일이 줄어 캐스케이드 전체가 싸집니다). 우려했던 FPR 열세는 운영점에서 재현되지 않았습니다(양쪽 모두 0.0014). gray 의 실패는 규칙 탓이 아니라 **1차 모델의 확신도 품질** 탓이며 — 이 구조의 성패는 게이트 품질이 좌우한다는 사실을 그대로 보여줍니다(gray match-teacher 의 speedup 은 1.11배로, 캐스케이드를 쓸 이유가 사라집니다).

4. RQ2 — 회피 강건성과 비용 기반 공격 (논문 5장)

4.1 위협 모델과 공격 설계

블랙박스 · 모델 무관 · 의미보존 표면 변형만 사용합니다. feature-space 와 problem-space 를 구분하며 **problem-space 가 주 실험**입니다 — 주석 삽입 `/**/`, 대소문자 랜덤화, 공백/탭 치환, URL/이중 인코딩, 동치 SQL 토큰 치환, XSS `String.fromCharCode` 등 **14종**을 누적 $k=1.5$ 로 적용합니다. 생성된 변형은 **실제로 동작하는 페이로드**입니다.

주 지표는 **any-misclass**(변형 하 오분류율)입니다. 원래 주 지표였던 benign-evasion(공격 → Normal)은 **5모델 전부 ~0**(0.0000~0.0022)이라 판별력이 없기 때문입니다(F8). 이 교체는 방어 실험을 돌리기 **전에** 문서로 고정했습니다.

⚠ **any-misclass 를 주 지표로 삼는 것의 정당성 — 심사에서 반드시 나올 질문입니다.** any-misclass 는 WAF 우회가 아닙니다(SQLi → XSS 로 틀려도 탐지·차단은 됩니다). 근거 3개를 미리 세워 둡니다.

- 1. **운영 비용이 실재합니다.** 공격 유형은 대응 playbook · 차단 규칙 · 포렌식 분류를 가릅니다. SQLi 를 XSS 로 라벨링하면 잘못된 대응이 자동 실행됩니다 — 탐지는 되지만 대응이 틀립니다.
- 2. **결정 경계 불안정성의 프록시입니다.** 의미보존 표면 변형에 예측이 흔들린다는 것은 그 표현이 표면 토큰 shortcut 에 의존한다는 직접 증거이며, 더 강한 공격에 대한 **선행 취약 지표**입니다.
- 3. **유일하게 변별력이 있는 강건성 측입니다**(모델 간 2.6배 차이).

한계도 같은 문단에 씁니다 — 이 축의 주장은 **"표현 안정성을 얼마의 비용으로 살 수 있는가"**이지 **"WAF 가 더 안전해졌다"**가 아닙니다.

4.2 강건성 서열 — 캐스케이드는 단일 CNN 보다 이미 강건하다

모델 / 운영점	any-misclass	benign-evasion	위치
char-CNN 단독	0.2239	0.0001	가장 강건(2차 단독)
캐스케이드 gray 1차 $\tau=0.9998$	0.3641	0.0001	τ 로 두 단독 사이를 이동
캐스케이드 gray 1차 $\tau=0.5678$	0.5557	0.0009	
캐스케이드 RGB 1차(제안 구성) base	0.4707	0.0001	제안 모델의 기준선
gray CNN 단독	0.5944	0.0022	가장 취약(1차 단독)
RGB CNN 단독	0.5981	0.0006	

표 4. 방어 없음 상태의 any-misclass ($k=5$, 낮을수록 강건). 캐스케이드 두 점은 운영점 τ 가 다른 같은 시스템이다.

기여 C2: τ 하나가 clean 정확도뿐 아니라 공격 하 강건성에서도 두 단독 모델 사이를 연속 이동합니다 ($0.364 \leftrightarrow 0.556$). 즉 §2 에서 주장한 "연속 스펙트럼"이 clean 측에만 성립하는 것이 아닙니다. 다만 $\tau=0.9998$ 에서도 char-CNN 단독(0.224)에 못 미치는데, 원인은 넘기지 않은 21.6% 입니다 — 1차가 확신했기에 남은 샘플이지만 변형되면 틀립니다. "확신도가 높다"가 "변형에 강하다"를 뜻하지 않습니다.

⚠ 수치 인용 주의(과거 문서와의 불일치) — 회피 수치 0.594 / 0.5944 / 0.0022 는 **gray** 실측입니다. **RGB** 기준선은 **0.5981**(단일 CNN), **RGB 캐스케이드 기준선은 0.4707**입니다. 캐스케이드에서는 채널이 1차 확신도 분포를 바꾸고 그것이 곧 게이트 동작이라, gray(0.3641)와 RGB(0.4707)의 차이가 **10.66pp** 로 큼니다. gray 수치를 기준선으로 쓰면 방어 효과 판정이 10pp 이상 왜곡됩니다.

4.3 ★ 비용 기반 공격 — 캐스케이드 고유의 공격면 (기여 C3)

운영점	any-misclass	esc (clean)	esc (공격 $k=5$)	비용 증가	결과
$\tau=0.9998$ (match-teacher)	0.3641	78.36%	79.96%	1.02배	증가 여지 없음
$\tau=0.5678$ (budget 5%)	0.5557	2.54%	5.50%	2.17배	예산 5% SLA 붕괴

표 5. 회피 변형($k=5$) 하에서의 에스컬레이션 변화. 정확도 지표로는 절대 보이지 않는 열화다.

비용 기반 공격은 실재합니다. 저에스컬레이션 운영점에서 공격은 정확도를 흔드는 것과 동시에 2차 호출을 2.17배로 늘려 SLA 를 깹습니다. 반대로 이미 78% 를 넘기는 운영점은 증가 여지가 없어 1.02배에 그칩니다 — 즉 **비용 공격 표면은 캐스케이드가 싸게 돌아갈수록 커집니다.** 완화책으로 에스컬레이션 비율을 운영 지표로 모니터링하고 상한 규를 두는 방안을 논문 고찰에 씁니다.

⚠ 주장 강도 — 선행 대조로 하향된 항목입니다(docs/12 §4.1).
 ✖ 쓰면 안 되는 말: "새로운 공격면을 발견했다"
 ✔ 쓸 말: "DeepSloth(ICLR 2021 Spotlight)가 보인 slowdown 공격이 웹공격 탐지 캐스케이드에서도 성립함을 실측했다. 단 DeepSloth 는 픽셀 단위 gradient 교란(feature-space · 화이트박스)인 반면, 본 연구는 모델 접근 없이 실제로 동작하는 페이로드 변형(problem-space)만으로 2.17배를 달성했다."
 → 이 재서술이 오히려 더 강합니다. 실전 WAF 우회 기법(주석 삽입 · URL 인코딩)만으로 성립하기 때문입니다.

5. RQ3 — 방어: 학습형 방어의 패배 (논문 5장)

5.1 방법론적 핵심 — seen / held-out 변형 분할

학습에 쓴 변형으로 평가하면 "본 적 있는 공격을 막았다"는 자명한 결과가 나옵니다. 변형 14종을 4계열로 묶고 평가 전용(held-out) 계열을 남깁니다.

계열	변형	실측 위력	S-0 (seen)	S-A (leave-encoding-out)
E. 인코딩	<code>url_encode</code> , <code>double_url_encode</code> , <code>xss_html_entity</code> , <code>xss_decimal_entity</code>	압도적 ($\Delta+0.59\sim0.62$)	학습	held-out
C. 대소문자	<code>random_case</code> , <code>xss_tag_case</code>	미미	학습	학습
W. 공백·주석	<code>space_to_comment</code> 외 4종	미미(1종 예외)	학습	학습
S. 구문 동치	<code>sqli_logical_equiv</code> 외 2종	미미	학습	학습

표 6. 변형 계열과 분할 시나리오. E 계열은 S-A 에서 train 에도 val 에도 넣지 않는다(val 에 넣으면 조기 종료를 통해 간접 누수).

S-0 은 낙관 상한(방어가 원리적으로 얼마나 줄일 수 있나), S-A 가 진짜 질문(미지의 강한 벡터에 일반화되나)입니다. 두 값의 격차 자체가 결과입니다. 증강은 치환식입니다 — 변형본을 추가(append)하면 공격 클래스만 불어나 FPR 이 방어 효과가 아니라 분포 변화 때문에 움직입니다. 균형화 이후 각 공격 클래스에서 비율 p 만큼을 변형본으로 바꿔치기해 train 크기·클래스 비율을 불변으로 둡니다.

F11 — 동요를 만드는 것은 사실상 3종뿐입니다: `space_to_comment` +0.635, `url_encode` +0.584, `double_url_encode` +0.572. 그중 2종이 S-A 의 held-out 이라 S-A 는 배울 것이 거의 없습니다. 이 점이 아래 결과 전체를 설명합니다.

5.2 실측 — any-misclass (k=5)

구성	캐스케이드(제안)	Δ	단일 RGB CNN(기준)	Δ	char-CNN	Δ
base (방어 없음)	0.4707	—	0.5981	—	0.2239	—
advtrain S-0 p0.25	0.0318	-43.89	0.1955	-40.26	0.0271	-19.68
advtrain S-0 p0.5	0.0287	-44.20	0.2029	-39.52	0.0187	-20.52
advtrain S-A p0.25	0.2780	-19.27	0.4693	-12.88	0.1176	-10.62
advtrain S-A p0.5	0.4227	-4.80	0.5704	-2.77	0.1383	-8.56
norm (B, 비학습 대조군)	(미측정)	—	0.2112	-38.69	0.0650	-15.89

표 7. 적대적 증강(A) 8셀 + 정규화 방어(B) 2셀 실측. 괄호 안은 base 대비 절대 변화(pp).

⚠ norm 행을 advtrain S-0 과 나란히 읽지 마십시오. S-0 은 14종 변형을 전부 학습에서 본 구성이고, norm 은 변형을 하나도 보지 않은 규칙 기반입니다. 배치 현실(미지 변형)에 맞는 비교 상대는 S-A 입니다.

5.3 판정 — 실측 전에 고정한 기준 그대로

#	기준(사전 고정)	판정	근거
H3-1	any-misclass ≥ 10 pp 절대 감소 = 효과 있음	✅ 효과 있음	S-0 에서 40~44pp 감소 = 기준의 4배 이상. 단일 split 노이즈 (± 0.11 pp)와 자릿수가 달라 CV 불필요
H3-2	clean MCC 하락 ≤ 1 pp and FPR 상승 ≤ 0.5 pp	⚠️ 표현에 따라 갈림	캐스케이드 전 구성 통과(MCC 최대 -0.12 pp, FPR 0.00 pp) · char-CNN 4/4 통과 · 단일 RGB CNN 만 2/4 실패(S-0 $\rho 0.5 -1.86$ pp, S-A $\rho 0.25 -3.81$ pp · FPR $+0.69$ pp)
H3-3 ★	S-A 감소폭 $\geq$ S-0 감소폭 의 50% → "일반화"	❌ 본 변형 암기(이미지) / ⚠️ 경계선상 충족(텍스트)	캐스케이드 43.9% · 10.9% / 단일 RGB CNN 32.0% · 7.0% → 미달. char-CNN 54.0%($\rho 0.25$) · 41.7%($\rho 0.5$)
H3-4 ★	advtrain 이 norm 을 못 이기면 그 사실을 헤드라인으로	❌ 학습형 방어의 패배	미지 변형 조건(S-A) 대비 norm 이 RGB CNN 0.2112 vs 0.4693/0.5704, char-CNN 0.0650 vs 0.1176/0.1383 로 전부 우세

표 8. 가설 판정. 기준은 실험 전에 문서로 못 박았고 사후 조정하지 않았다.

★ H3-4 가 이 Phase 의 헤드라인입니다 — 그리고 이것은 사전에 약속한 서술입니다.

설계 문서가 "advtrain 이 norm 을 못 이기면 그 사실을 헤드라인으로 낸다"고 실험 전에 못 박아 두었고, 실측이 정확히 그 경우였습니다. 8셀을 학습해 얻은 것을, 학습이 전혀 없는 규칙 기반 전처리가 미지 변형에서 더 잘 막습니다. 원인은 F11 과 정확히 맞물립니다 — 동요의 주범 3종 중 2종이 인코딩 계열이므로, 인코딩은 학습이 아니라 정규화로 막는 종류입니다.

단, 공짜는 아닙니다. 이미지 표현에서 정규화 방어는 clean MCC 를 1.04pp 깎아(0.9725 → 0.9621) H3-2 기준을 넘겼고, benign-evasion 도 유일하게 base 보다 올랐습니다 (0.0006 → 0.0016). 텍스트 표현에서는 그 대가가 0 입니다 (char-CNN 0.9944, +0.04pp).

5.4 ★ 캐스케이드는 방어 비용을 '정확도'가 아니라 '지연'으로 지불한다 (F10)

구조	clean MCC 하락	FPR 변화	공격 하 에스컬레이션
단일 RGB CNN	최대 -3.81 pp	$+0.69$ pp	(해당 없음)
캐스케이드(제안)	최대 -0.12 pp	0.00pp	34.97% → 최대 69.52%

표 9. 같은 방어를 걸었을 때 두 구조의 대가가 다른 축에 나타난다.

캐스케이드는 1차가 흔들려도 2차 char-CNN 이 받아내므로 정확도를 거의 잃지 않습니다. 대신 1차 확신도가 낮아진 만큼 2차 호출이 늘어 추론 비용이 최대 2배가 됩니다. 방어 효과가 큰 구성일수록 에스컬레이션도 큼니다.

논문 서술 주의 — 이는 정확도 지표만 보면 "공짜 방어"로 오독되는 열화입니다. §4.3 의 비용 기반 공격과 직결되며, 캐스케이드를 제안 모델로 세웠기 때문에 비로소 드러난 축입니다. 단일 CNN 만 방어했다면 이 결과는 논문에 존재하지 않았습니다.

6. Phase 12 확장 3안 — 무엇을 채택했고 무엇을 버렸는가

선행연구 전수 대조 결과 두 가지가 드러났습니다. ① 선행 논문들의 정확도 개선폭 기준선이 낮고 ($+0.11 \sim 1.27$ pp 를 "surpassing existing benchmarks"로 서술), 8편 중 유의성 검정을 한 탐지 논문은 0편입니다. ② 8편 중 회피/적대적 실험을 한 논문도 0편입니다. 그런데 그 축에서 우리가 가진 것은 아직 "제안 모델도 흔들린다"는 음성 결과뿐이었습니다. → 그래서 캐스케이드에만 성립하는 능력 3안을 시도했습니다.

방안	무엇을 시도했나	판정	핵심 수치	논문에서의 자리
M1 표현 불일치	1·2차 예측 불일치를 회피 시도 신호로	⚠ 보조 지표로 하향	ROC-AUC 0.764 (기준 0.90), 커버리지 35.5%, 추가 비용 0	RQ6 부분 답 · 기여 C5
M2 인코딩 불변 채널	%XX · &#NN 해독 후 문자 클래스를 G 채널로	⚠ 단독 미채택 / 증강 결합 채택	단독 0.4707 → 0.5552 (악화) · 증강 결합 S-A 0.4227 → 0.1833	RQ3 보강 · 기여 C6-C7
M4 다단 조기 종료	1차 비용 C_cnn 자체를 낮춰 속도 천장 돌파	❌ 미채택	연산 18% 생략했는데 시간 +3.3%, speedup 5.07 → 3.99배	한계로 정직 보고
(M3) CAM 크롭	—	● 미착수	—	Phase 13 후보

표 10. Phase 12 종결 요약. 판정 기준은 전부 실측 전에 고정했고 사후 조정하지 않았다.

6.1 ★ M2 — 인코딩 불변 채널: 단독은 실패, 증강과 결합하면 F9 를 뒤집는다

착안: 적대적 증강이 미지 변형에 실패한 원인(F9)은 F11 입니다 — 동요를 만든 3종 중 2종이 인코딩 계열이고, S-A 에서 그것이 held-out 이라 **학습이 배울 것 자체가 없었습니다**. 학습으로 못 얻는 불변성은 구조로 넣습니다. 채널 인코더 레지스트리에 `normalized_char_class` (약어 `nc`)를 추가해 $R = \text{raw_byte} / G = \text{정규화 문자 클래스} / B = \text{local_entropy}$ 로 둡니다. **R 채널(raw)**을 그대로 두는 것이 정규화 방어(B안)와의 결정적 차이입니다 — 원본 뷰와 정규화 뷰를 동시에 보므로 **"인코딩을 썼다는 사실"** 자체도 신호로 남습니다.

구성	캐스케이드 cc	캐스케이드 nc(M2)	Δ	단일 CNN cc	단일 CNN nc(M2)	Δ
base (방어 없음)	0.4707	0.5552	+8.45pp ❌	0.5981	0.6550	+5.69pp ❌
advtrain S-0 p0.25	0.0318	0.0410	+0.92pp	0.1955	0.2025	+0.70pp
advtrain S-0 p0.5	0.0287	0.0179	-1.08pp	0.2029	0.0749	-12.80pp
advtrain S-A p0.25 ★	0.2780	0.1548	-12.32pp	0.4693	0.3720	-9.73pp
advtrain S-A p0.5 ★	0.4227	0.1833	-23.94pp	0.5704	0.4137	-15.67pp

표 11. M2 실측 — any-misclass (k=5). cc = 기존 문자 클래스 채널, nc = 인코딩 불변 채널.

읽는 법이 전부입니다. M2 는 단독으로는 회피를 더 쉽게 만들고(base 두 줄 모두 악화), **적대적 증강과 결합했을 때만** 이득을 냅니다. 그리고 그 이득은 **미지 변형 조건(S-A)에 몰려 있습니다** — 정확히 F9 가 실패했던 지점입니다.

구성	p0.25	p0.5
캐스케이드 cc (기준)	43.9%	10.9%
캐스케이드 nc (M2)	77.9% ✅	69.2% ✅
단일 CNN cc (기준)	32.0%	7.0%
단일 CNN nc (M2)	62.6% ✅	41.6% ❌

표 12. H6-1 판정 — S-A 감소폭 / S-0 감소폭 (기준 50%). F9 가 처음으로 흔들린 지점.

⚠ **주장 강도** — 정확한 문구는 **"부분적으로 뒤집었다"**입니다. 단일 CNN p0.5 의 41.6% 는 설계 문서가 **"40~50% 경계면 5-fold CV 로 재판정"** 하라고 지정한 밴드 안입니다. 캐스케이드 두 셀(69.2% · 77.9%)은 밴드 위라 단일 split 로 보고 가능하나, 비율에는 단일 split 규칙이 명시돼 있지 않습니다. → 논문에는 **"캐스케이드에서 기준을 넘었다"**로 쓰고, 강한 주장이 필요하면 CV 를 겁니다. **"F9 를 해결했다"** ❌ / **"F9 를 부분적으로 뒤집었다"** ✅

6.2 ★★ 왜 단독으로는 실패했나 — 설계 가설 자체가 틀렸다 (기여 C7)

M2 의 전제는 **"인코딩 변형의 위력은 바이트 값이 바뀌는 데서 온다"**였습니다. 실측이 그 전제를 기각했습니다.

변형	실제 변형된 비율	길이 배율	any-misclass
space_to_comment	78.0%	1.36×	0.6542
url_encode	99.9%	1.44×	0.6029
double_url_encode	99.9%	1.88×	0.5916
xss_html_entity	44.7%	1.12×	0.0049
random_case	94.5%	1.00×	0.0294

표 13. 회피 위력은 '값'이 아니라 '길이'를 따라간다 (사후 분석, 공격 21,907건).

위력 있는 변형 3종은 **전부** "대다수 샘플을 건드리면서 길이를 1.36배 이상 늘리는" 변형이고, 길이를 보존하는 변형 8종은 **전부** 0.08 이하입니다. 이미지화는 바이트열을 48×48 래스터에 **행 우선**으로 깔기 때문에, 길이가 늘면 같은 내용이라도 **2차원 배치가 통째로 밀립니다**(중앙 길이 267B → 370B). **채널 값을 고쳐도 배치는 못 고칩니다.**

방법	무엇을 되돌리나	any-misclass
M2 (인코딩 불변 채널)	채널 값 만 — 길이·배치는 그대로	0.6550
B안 norm (입력 정규화)	텍스트를 디코딩 → 값과 길이·배치 모두	0.2112

표 14. 결정적 대조 — 두 방법의 차이가 변수를 정확히 분리한다.

되돌려야 하는 것은 값이 아니라 배치였습니다. 이는 §5.3 의 "*인코딩은 학습이 아니라 정규화로 막는 종류*"를 한 단계 더 밀어붙인 결과입니다 — 정규화가 이기는 이유는 디코딩 자체가 아니라 **디코딩이 길이를 되돌리기 때문**입니다.

그런데 왜 증강과 결합하면 이기는가? clean 학습 데이터에서 `nc` 와 `cc` 가 다른 픽셀은 **0.20%뿐**입니다(train 105,598건 기준). 즉 증강이 없으면 모델은 이 채널이 무엇을 하는지 배울 기회 자체가 없고, 공격 시점에는 세 채널이 서로 어긋난 처음 보는 입력이 됩니다(그래서 base 가 오히려 약화). 증강이 들어가야 "인코딩 하에서는 G 채널이 믿을 만한 뷰"라는 것을 학습하고, 그 순간 **held-out 인코딩에도 전이**됩니다. → 불변성은 표현에 넣는 것만으로는 작동하지 않고, 그것을 쓰라고 가르치는 데이터가 함께 필요합니다.

⚠ M2 를 켜면 비용 기반 공격(C3)의 표면이 넓어집니다 — 반드시 병기합니다. 방어가 실제로 듣는 구성(S-A p0.5, -23.94pp)일수록 공격 하 에스컬레이션이 35.39% → **77.71%** 까지 오릅니다. clean 에서도 1차 확산도가 낮아져 speedup 이 2.52배 → 2.13배로 떨어집니다. **강건성과 지연이 정면으로 상충합니다.**

6.3 M1 — 표현 불일치를 회피 시도 신호로 (RQ6, 보조 지표)

착안: clean 에서는 1.2차의 탐지집합이 **완전 일치(F4)**하는데, 변형 k=5 에서는 any-misclass 가 char-CNN 0.224 vs RGB CNN 0.598 로 **격렬하게 갈립니다**. 지금까지 이 격차는 "캐스케이드의 약점"으로만 읽혔습니다. 뒤집으면 **잡음(clean)이 거의 없고 신호(변형)가 큰 탐지기**입니다. 캐스케이드에 4-class 판정과 별개의 세 번째 출력 $\text{tampering_score} = D(p_1, p_2)$ 를 답니다.

모드 A(비용 불변) — 에스컬레이션 부분집합에서만 판정하므로 **추가 비용이 정확히 0** 입니다. 이것이 M1 의 **유일**한 기여이며, 주 점수함수(js_divergence) · 주 모드 · 음성 집합 · 임계값 규칙은 전부 **실측 전에** 고정했습니다.

구성	k=1	k=3	k=5(주)	커버리지	clean Normal FPR	판정
base	0.6401	0.7374	0.7644	0.3552	0.0301	H5-1 보조 지표 / H5-2 미충족
방어(advtrain S-0 p0.5)	0.5522	0.5920	0.5980	0.6532	0.0149	H5-1 기각 / H5-2 미충족

표 15. M1 주 지표 (모드 A · js_divergence · 음성집합 = 같은 페이로드의 clean 판본). 새 학습 0, 실행당 약 90초.

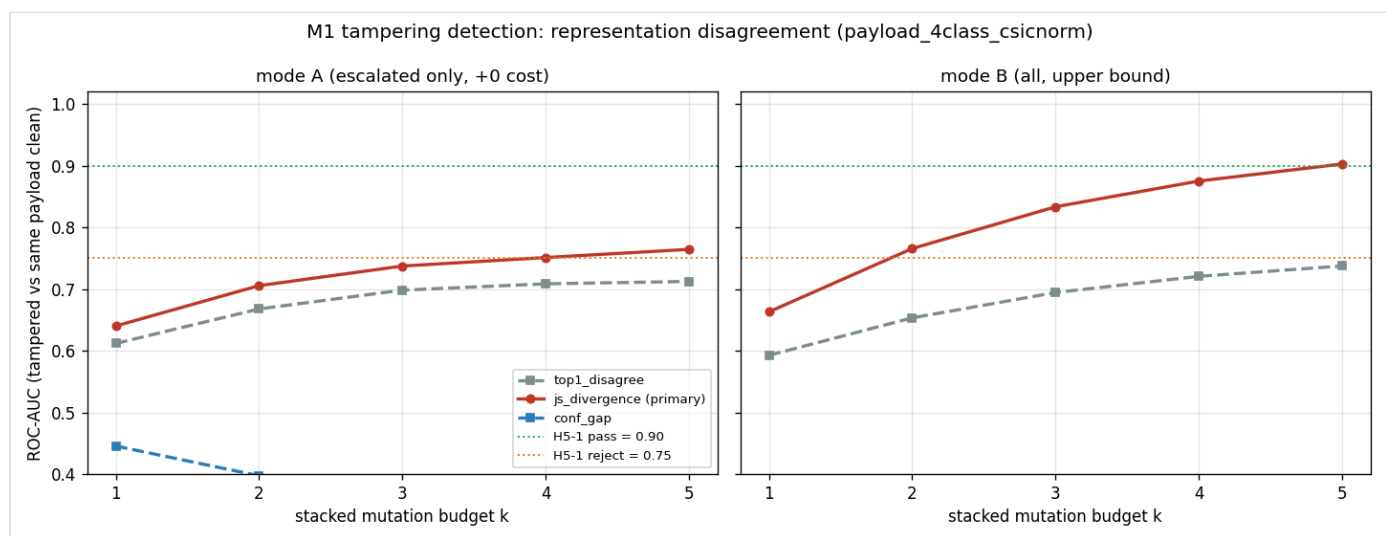


그림 2. τ 스윙에 따른 판별력 변화(base 구성, k=5). τ 를 올리면 커버리지와 ROC-AUC 가 함께 오르지만 그만큼 2차를 더 돌리므로 비용도 오른다 — "공짜 판정"과 "판별력"은 같은 손잡이의 양 끝이다. (docs/figures/attacks/tamper_auc_..._cascade_rgb_bal.png)

τ	0.878	0.964	0.993	0.997(운영)	0.9999	1.01(=전량)
커버리지	0.092	0.162	0.277	0.355	0.644	1.000
ROC-AUC	0.522	0.669	0.748	0.764	0.818	0.903

표 16. τ 스윙 (base, k=5, 추가 비용 0 — 확률은 이미 계산돼 있다).

M1 의 핵심 결론: H5-1 의 0.90 기준을 넘는 지점은 **커버리지 100%(=모드 B)** 하나뿐입니다(0.9028). 그런데 그 점은 비용이 융합형과 같아져 M1 의 유일한 기여가 사라집니다. → "공짜이면서 효과 있음"은 이번 실측에서 성립하지 않았습니다.

- **H5-2 가 깨진 지점은 커버리지가 아니라 오탐입니다.** 커버리지는 base 35.5% / 방어 65.3% 로 30% 기준을 양쪽 다 넘겼고, 걸린 것은 clean Normal FPR 3.01%(기준 $\leq 1\%$)뿐입니다. ⚠️ 그 3.01% 의 실체는 4건입니다 — 모드 A 의 판정 대상은 에스컬레이션된 샘플뿐이라 test Normal 722건 중 133건만 남고, 임계값도 val 의 같은 크기 부분집합에서 1% 분위수로 골랐습니다. **원인은 신호가 아니라 추정 표본 크기**이며, 기준을 사후 완화하지 않고 미충족으로 보고하되 한계로 명시합니다.
- ⚠️ **방어가 신호를 지웁니다**(AUC 0.7644 → 0.5980, -16.6pp). 적대적 증강이 두 모델을 같은 방향으로 강건하게 만들자 두 표현이 더 이상 싸우지 않습니다. 즉 **M1 과 적대적 증강 방어는 서로를 잡아먹습니다**. Feature Squeezing 계열 선행에서도 보고되지 않은 조합이라 **한계이자 관찰로 함께 서술**합니다.

G3 상환 — 변형 조건 상보성 (base, 공격 21,907건)

조건	기준	둘 다	1차만	2차만	둘 다 실패
clean	detection	21,865	8	34	0
변형 k=5	detection	21,892	2	13	0
변형 k=5	correct	8,359	445	8,644	4,459

표 17. 두 기준으로 낸 상보성. `detection` = Normal 로 안 샘, `correct` = 진짜 클래스 일치.

⚠ 이 표의 서술 방식이 논문 정직성의 핵심입니다. `detection` 기준으로는 변형 하에서도 순증이 13건뿐입니다 — 즉 변형은 공격을 Normal 로 넘기지 못하며 **미탐이 아닙니다**. 격차는 전부 `correct` (유형 귀속)에서 나며 2차만 맞춘 것이 8,644건입니다. → 논문에는 반드시 "**더 많이 탐지한다**"가 아니라 "**공격 유형을 옳게 귀속한다**"로 씁니다. 운영 비용 근거 (대응 playbook · 차단 규칙이 유형으로 갈림)를 같은 문단에 인용합니다.

6.4 M4 — 다단 조기종료: 미채택 (부정 결과, 논문에 그대로 실습니다)

착안: 캐스케이드 비용식 $C(r) = C_{\text{cnn}} + r \cdot C_{\text{charcnn}}$ 에서 지금까지의 모든 논의는 r 만 건드렸고 C_{cnn} 은 상수로 뒀습니다. 게이트를 완벽히 고쳐도 (oracle $r=1.87\%$) **7.29배가 천장**이고 그 원인은 C_{cnn} 이 바닥이기 때문입니다. 1차 CNN 내부에 조기종료 헤드를 달아 그 천장을 깨려 했습니다. **마스킹이 아니라 축소 배치**로 구현했습니다(그래야 자연이 실제로 줍니다. 근거는 eval 모드 BatchNorm 이 running stats 를 쓴다는 것이며, 단위 테스트로 고정했습니다).

단계	1차 MCC	1차 지연	캐스케이드 speedup (budget 10%)
① 기존 RGB CNN(기준)	0.9725	6.58 μs	5.07배
② <code>cnn_ee</code> , 조기종료 꿈	0.9554	9.93 μs	4.07배
③ <code>cnn_ee</code> , 조기종료 컴(임계값 0.870)	0.9552	10.25 μs	3.99배

표 18. M4 판정 — 두 번의 손실. 원인이 하나가 아니다.

- ①→② 가 손실의 대부분입니다. 조기종료를 켜기도 전에 이미 졌습니다. 보조 헤드 2개를 달고 보조 손실로 함께 학습한 것만으로 1차 MCC **-1.71pp**, 비용도 올랐습니다.
- ②→③ 이 H7-1 을 직접 깬다. 연산을 18% 생략했는데 시간은 6% 늘었습니다(GPU 기준).
- 부작용 하나 더: `match-teacher` 에서 에스컬레이션이 27.7% → 90.0% 로 폭증했고 그 운영점의 speedup 은 0.96배 — char-CNN 단독보다 느립니다. 1차 정확도 하락은 C_{cnn} 뿐 아니라 r 까지 악화시킵니다.

★ 왜 안 줄었나 — 구현 결함이 아니라 하드웨어 영역이 원인입니다.

같은 가중치로 종료 임계값만 바뀌가며 GPU-CPU 양쪽에서 잤습니다. CPU 는 깊이에 따라 단조로 줄어듭니다(-56% → -30% → -8.7% → -0.9%) — **축소 배치 구현은 연산을 실제로 생략하고 있으며 구현 결함이 아닙니다**. 그런데 GPU 는 채택 지점에서 오히려 느려집니다. 종료 루프의 파이썬 분기가 GPU→CPU 동기화를 강제하고 boolean 인덱싱이 텐서를 새로 만드는데, 94K 파라미터 모델의 배치 1회가 10 μs 남짓인 영역에서는 이 고정비가 지배합니다. 전원 1블록(연산 -91%)에서조차 시간은 -46% 밖에 안 줄어듭니다 — 이 모델은 GPU 에서 연산 병목이 아니라 커널 런치·동기화 병목입니다.

H7-2 는 이 하드웨어에서 어떤 임계값으로도 도달 불가능했습니다. 6.5배가 되려면 C_{cnn} 을 -53% 해야 하는데, 조기종료의 이론적 바닥(정확도를 완전히 포기한 1블록 CNN)조차 -46% 입니다. 실패는 임계값을 잘못 골라서가 아니라 **비용 바닥이 목표 위에 있어서**이며, 스윙해도 뒤집히지 않습니다.

논문 서술: "조기종료가 무용하다"가 아니라 "**얇은 CNN 1차는 GPU 에서 이미 바닥이라 더 뺄 것이 없다**"로 씁니다. 이는 기여 C1(1차가 이미 충분히 빠다)의 근거를 오히려 강화합니다. 선행 연구(ACM SAC 2025 early-exit IDS 등)가 조기종료로 이득을 본 것은 CPU/엡지 배포나 1차가 훨씬 큰 모델(BERT 급)의 영역입니다.

7. RQ6 정리 — 캐스케이드에서만 얻어지는 능력 (부분 답)

RQ6("1차와 2차의 표현 간 불일치 자체가 회피 시도의 탐지 신호가 되는가")의 답은 **부분 답**입니다. 추가 비용 0 으로 신호는 나오지만(ROC-AUC 0.764, 커버리지 35.5%) 사전 기준 0.90 에 미달합니다. **보조 지표로만** 보고하며, 단일 모델은 원리적으로 이것을 할 수 없다는 점(두 표현을 동시에 유지하는 구조에서만 공짜로 얻어짐)이 **제안 모델이 캐스케이드여야 하는 두 번째 이유**입니다(첫 번째는 비용).

8. RQ4 — 내용 기반 탐지의 경계 (별도 축, 논문 6장)

- **F12** — 단계적 신호 소멸 실험(평문 → 인코딩 → 고정키 XOR → AES)에서 **고정키 XOR 은 F1 0.9445 를 유지하다가 AES 에서 0.3965 로 붕괴**합니다. → 탐지를 죽이는 것은 엔트로피가 아니라 키 랜덤화입니다.
- ⚠ **초점 재조정**: "신호가 흐름으로 이동한다"는 결론은 **선행이 이미 도달해** 있습니다 (Kutlimuratov et al. 2026 은 처음부터 흐름 메타데이터만 사용, 470k req/s). → 우리 기여는 도착지가 아니라 **경계의 정량 증명(F12)** 으로 좁힙니다.
- **G6 미결** — RQ4b(USTC-TFC2016 흐름 트랙)는 F1 0.9999 로 **포화**되어 판별력이 없습니다. **부록 강등 vs host 단위 분할 재실험**은 교수님 확인이 필요한 항목입니다.

9. 선행연구 대조 — 차별화의 근거 (논문 2장)

비용 개선을 헤드라인으로 삼은 캐스케이드 웹공격 탐지 논문이 이미 존재합니다(Tasdemir et al. 2023, SQLi, 20배). 프레이밍이 거의 같고 배수는 저쪽이 큼니다. 숨기지 말고 먼저 인용하고 대조합니다.

축	Tasdemir 2023	본 연구
1차/2차 표현	텍스트 → 텍스트 (동종)	이미지 → 텍스트 (이종) ★
라우팅 기준	예측 클래스(양성이면 승급)	예측 불확실성($\max \text{softmax} < \tau$) ★
운영점	고정 1개	τ 스윙 연속 곡선, 양 끝점이 두 단독 모델로 수렴(단위 테스트로 고정) ★
과제	SQLi 이진	SQLi / XSS / Cmdl / Normal 4-class
게이트 품질	미분석	oracle 대비 14.8배 과잉 정량화 ★
회피 강건성	없음	RQ2 / RQ3 전 축 ★
통계 검정	없음	5-fold CV + Holm 보정
속도 배수	20× (vs BERT)	5.07× (vs char-CNN)

표 19. Tasdemir et al. 2023 (arXiv:2312.13041) 대조표 — 이 표가 차별화의 근거다.

⚠ 배수를 직접 비교하지 마십시오. 2차 기준 모델이 다릅니다(BERT vs char-CNN). BERT 가 훨씬 무거우므로 분모가 커서 배수가 크게 나옵니다. "우리가 5.07배라 저쪽 20배보다 못하다"는 식의 서술도, 그 반대도 성립하지 않습니다. 기준 모델을 명시해 병기합니다.

라우팅 차이가 만드는 실질적 귀결: Tasdemir 방식은 공격처럼 보이는 트래픽을 보내면 그대로 2차가 호출되어 비용 기반 공격에 구조적으로 더 취약합니다(공격자가 확신도를 조작할 필요조차 없습니다). 우리 확신도 게이트는 1차의 확신도를 떨어뜨려야 하므로 한 단계 더 어렵습니다. 그럼에도 problem-space 변형만으로 2.17배를 달성했습니다. → 논문 서술은 "클래스 기반 라우팅과 불확실성 기반 라우팅은 비용 공격면의 크기가 다르며, 후자가 더 좁지만 0 은 아니다" 로 쓰면 대조와 발견이 한 문단에 들어갑니다.

라인	대표 연구	우리 축과의 관계
이미지 기반 악성코드 탐지	Nataraj et al., VizSec 2011 ★	이미지화 계열의 원점. 바이트-플롯 = 우리 R 채널의 근거
소스코드 이미지화	VulCNN, ICSE 2022 ★ / Appl. Sci. 2025 assembly-RGB	채널에 구문 정보를 넣는 계열 → G 채널의 근거
텍스트 기반 웹공격 탐지	Tadhani et al., Sci. Rep. 14:1803 (2024)	비교 대상(CNN+LSTM 하이브리드)
저비용 캐스케이드 탐지	Tasdemir et al. 2023 ★ / Sevri & Karacan 2022 / Kutlimuratov et al. 2026	● 최근접 선행 — 표 19 로 대조
조기종료	ACM SAC 2025 early-exit IDS · ACM CSUR 서버이	M4 의 선행(미채택이지만 관련연구에는 남김)
slowdown 공격	DeepSloth, ICLR 2021 Spotlight ★	● C3(F7)의 직접 선행 — 주장 하향 근거
표현 불일치 탐지	Feature Squeezing, NDSS 2018 ★	● C5(M1)의 직접 선행 — 주장 하향 근거
지표 선택 근거	Technologies 2026 14(1) 54 ★ / arXiv:2512.19203	불균형 데이터에서 MCC-PR-AUC 를 쓰는 근거
웹공격 회피 / 포이즈닝	WAF-A-MoLE / IVul	RQ2 위협모델 · 향후 연구

표 20. 관련연구 라인과 반드시 대조할 선행 (★ = 본문에서 인용-대조 필수). 전체 인벤토리 18편은 docs/12 §1.3.

10. 논문이 주장하는 기여 — 확정 문구와 금지어

아래 표는 선행 대조 결과 **과장을 걷어낸 최종 문구**입니다. 논문에는 **왼쪽 열을 그대로** 쓰고, 오른쪽 열의 표현은 **쓰지 않습니다**.

#	기여	✅ 쓸 문구	❌ 쓰면 안 되는 문구
C1	표현 이질적 캐스케이드	"이미지 표현과 텍스트 표현을 잇는 캐스케이드로, 같은 정확도를 5.07배 싸게 얻는다"	"이미지 기반이 더 정확하다"
C2	τ 연속 스펙트럼	" τ 하나가 clean 정확도뿐 아니라 공격 하 강건성에서도 두 단독 모델 사이를 연속 이동한다(0.364 $\leftrightarrow$ 0.556)"	—
C3	problem-space slowdown 공격	"DeepSloth 가 보인 slowdown 공격이 웹공격 탐지 캐스케이드에서도 성립함을 실측했다. 단 모델 접근 없이, 실제로 동작하는 페이로드 변형만 으로 2.17배를 달성했다"	"새로운 공격면을 발견했다"
C4	게이트 품질의 상한 정량화	"확신도 게이트는 oracle 대비 14.8배 과잉 공급한다 — 개선 여지의 상한을 못 박았다"	—
(C5)	표현 불일치 회피 탐지 ⚠️ 하향	"캐스케이드는 이미 2차를 돌리므로 추가 비용 정확히 0 (모드 A)으로 회피 시도의 보조 신호를 낸다(ROC-AUC 0.764, 커버리지 35.5%)"	"표현 불일치를 쓰는 것이 새롭다" · "회피 시도를 탐지한다" · "실용적이다"
(C6)	인코딩 불변 채널 + 증강 ⚠️ 조건부	"인코딩 불변 채널은 단독으로는 듣지 않지만, 적대적 증강과 결합하면 미지 인코딩 변형으로의 일반화를 10.9% $\rightarrow$ 69.2% 로 끌어올린다(캐스케이드, clean 대가 -0.02pp)"	"인코딩 불변 채널이 회피를 막는다" · "정규화 방어를 대체한다" · "F9 를 해결했다"(→ 부분적으로 뒤집었다)
C7	인코딩 회피의 작동 원리 진단	"이미지화 탐지에서 인코딩 회피의 위력은 바이트 값이 아니라 길 이 팽창에 따른 2차원 배치 밀림에서 온다 — 값만 되돌린 M2(0.6550)와 길이까지 되돌린 정규화(0.2112)의 대조가 변수를 분리한다"	"길이가 유일한 원인이다"(상관 0.856 은 강한 3점이 주도)

표 21. 기여 C1~C7 과 금지어. (C5~C6 는 조건부이므로 괄호로 표기)

11. 정직하게 함께 쓸 한계 (논문 7장)

- 이미지 단독은 텍스트 모델에 **유의하게 진다**(F1).
- clean 탐지 **상보성 이득 없음**(F4) — 캐스케이드의 기여는 탐지집합 확장이 아니라 비용 축소다.
- 적대적 증강은 **본 변형만 암기하며**(F9), 그 정도는 표현에 의존한다.
- 소규모 배포에서는 **char-CNN 단독으로 충분하다** — 캐스케이드가 필요한 것은 고처리량 환경이다.
- 적응형 공격 미수행**(G4) — 방어를 알고 재설계된 공격은 다루지 않았다. 방어 논문의 표준 비판 지점이므로 명시한다.
- 성능 차이는 **표면 토큰 포화** 상태의 트랙에서 측정됐다.
- 조기종료 1차(M4)는 시도했으나 미채택** — 얇은 CNN 1차는 GPU 에서 이미 커널 런치 병목이라 연산을 18% 생략해도 시간이 6% 늘었다. ViT/hybrid 와 같은 성격의 정직한 부정 결과이며, C1 의 근거를 오히려 강화한다.
- 적대적 증강 방어를 켜면 C5 의 불일치 신호가 지워진다**(ROC-AUC 0.764 $\rightarrow$ 0.598) — RQ3 방어와 RQ6 부가 능력은 서로를 잡아먹는다.
- 인코딩 불변 채널(M2)은 단독으로는 회피를 오히려 쉽게 만든다**(0.5981 $\rightarrow$ 0.6550) — clean 학습셋에서 이 채널이 기존 채널과 다른 픽셀은 0.20%뿐이라 증강 없이는 모델이 쓸 줄 모른다. "**불변성을 표현에 넣으면 공짜로 강건해진다**"는 직관에 대한 반례다.
- M2 를 켜면 비용 기반 공격(C3)의 표면이 넓어진다** — 공격 하 에스컬레이션 35.39% $\rightarrow$ **77.71%**. 강건성과 지연이 정면으로 상충한다.
- 캐스케이드 **norm**(B안) 판본은 **미측정** — M2 vs B안의 캐스케이드 수준 직접 대조는 불가능하며, 위 비교는 단일 RGB CNN 기준이다.

12. 단일 CNN p0.5 의 H6-1 41.6% 가 **경계 밴드(40~50%)** 안이다 — 강한 주장이 필요하면 5-fold CV 재판정이 필요하다.
13. **RQ4b(흐름 트랙)는 포화(F1 0.9999)로 판별력이 없다** — 부록 강등 여부는 미결(G6).

12. 논문 목차 매핑과 그림·표 인벤토리

논문 장	내용	이 문서	근거
1. 서론	배경 · 문제정의 · 기여 C1~C7	\$10	마스터 §11
2. 관련 연구	이미지화 탐지 / 텍스트 웹공격 탐지 / 캐스케이드·조기종료 저비용 탐지 (신설) / WAF 우회 · adversarial ML · slowdown	\$9	docs/12
3. 제안 방법론	이미지 변환(RGB 채널) · 캐스케이드 구조와 τ · 공격/방어 설계	\$2	docs/09 §2-§3
4. 실험 환경	데이터셋 · 구현 · 하이퍼파라미터 · 지연 측정 프로토콜 · lr 함정	\$2.2 · \$1.2	docs/09 §3.3, docs/08 §9.1
5. 실험 결과	RQ1 → RQ5 → RQ2 → RQ3 → RQ6 순	\$1 · \$3 · \$4 · \$5 · \$6 · \$7	docs/04-09-10-11
6. RQ4	내용 기반 탐지의 경계(별도 장 또는 절)	\$8	docs/06
7. 결론 · 한계 · 향후 연구	한계 13건 + 후속 과제(게이트 개선 · 배치 불변 이미지화 · 적응형 공격)	\$11	docs/11 §16.3

표 22. 논문 목차 ↔ 이 문서의 절 ↔ 근거 문서.

논문 위치	그림	파일
3장	페이로드 이미지화 샘플(RGB 3채널)	imaging/payload_4class_csicnorm_samples_text_raw_48_rgb.png
3장 / 6장	인코딩 불변 채널(nc) 샘플	imaging/payload_4class_csicnorm_samples_text_raw_48_rgb-rb-nc-le.png
5장 RQ1	5-fold CV 모델 순위(MCC)	models/cv_ranking_mcc.png
5장 RQ5	캐스케이드 정확도/지연 곡선(대표 운영점)	models/cascade_curve_..._rgb_b0.1_bal.png
5장 RQ5	캐스케이드 혼동행렬	models/cm_..._cascade_raw_rgb_b0.1_bal.png
5장 RQ2	모델별 회피 비교 (캐스케이드 2점 포함)	attacks/evasion_compare_payload_4class_csicnorm.png
5장 RQ2	누적 변형 예산별 회피율(제안 구성)	attacks/evasion_stacked_..._cascade_rgb.png
5장 RQ3	방어 arm 회피율(S-0 / S-A × ρ)	attacks/evasion_stacked_..._cascade_rgb_def-advtrain_*.png
5장 RQ3 / 6장	M2 채널 방어 arm 회피율	attacks/evasion_stacked_..._cascade_rgb-rb-nc-le_def-advtrain_*.png
5장 RQ6	tampering 판별력 · 커버리지 τ 스위프	attacks/tamper_{auc,coverage}_..._cascade_rgb_bal.png
6장 RQ4	엔트로피 스위프(평문 → 인코딩 → XOR → AES)	rq4/entropy_sweep_payload_4class.png

표 23. 논문에 넣을 그림 후보 (전부 docs/figures/ 에 추적·커밋되어 있음).

RQ2 evasion vulnerability across 7 detectors (payload_4class_csicnorm)

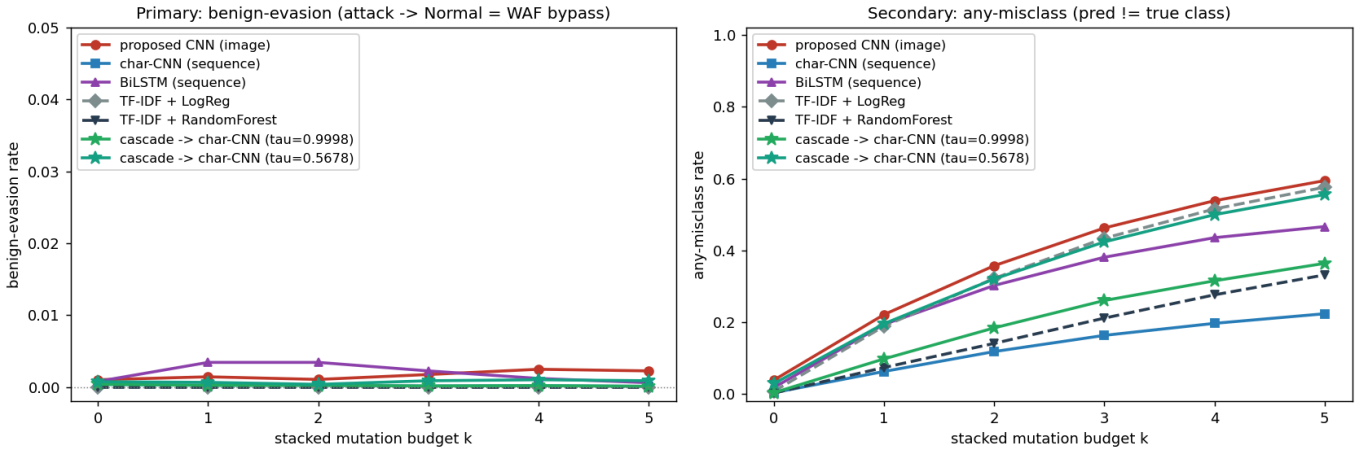


그림 3. 모델별 회피 비교(캐스케이드 운영점 2개 포함). 표현방식에 따라 any-misclass 가 2.6배 갈린다.

(docs/figures/attacks/evasion_compare_payload_4class_csicnorm.png)

13. 재현 절차 (논문 부록)

데이터·산출 JSON 은 용량 때문에 `.gitignore` 대상이며 **재생성으로 확보**합니다. 학습 GPU 는 1대뿐이므로 **순차 실행**이 원칙입니다.

```
# 0) 데이터 재생성 (seed=42 결정론, 인자 없이 전체 실행)
python src/data/preprocess.py
python src/imaging/build_image_dataset.py --track payload_4class_csicnorm --channels rgb

# 1) 두 단계 모델 학습 (캐스케이드는 재학습하지 않는다)
python src/models/train.py --model cnn --track payload_4class_csicnorm --balance --channels rgb
python src/models/train.py --model charcnn --track payload_4class_csicnorm --balance

# 2) 캐스케이드 평가 -  $\tau$  를 val 에서 확정 → test 1회 적용
python src/models/cascade.py --track payload_4class_csicnorm --balance --channels rgb
python src/models/cascade.py --track payload_4class_csicnorm --balance --channels rgb \
    --select budget --budget 0.10 # 대표 운영점

# 3) 회피 실험 (RQ2) - 같은 공격·같은 파이프라인, 대상 모델만 교체
python src/attacks/run_evasion.py --model cascade --track payload_4class_csicnorm --channels rgb
python src/attacks/compare_evasion.py --track payload_4class_csicnorm

# 4) 방어 (RQ3) - 적대적 증강 8셀 + 정규화 방어 2셀
for SPLIT in S0 SA; do for R in 0.25 0.5; do
    python src/models/train.py --model cnn --track payload_4class_csicnorm --balance \
        --channels rgb --defense advtrain --mutation-split $SPLIT --aug-ratio $R
    python src/models/train.py --model charcnn --track payload_4class_csicnorm --balance \
        --defense advtrain --mutation-split $SPLIT --aug-ratio $R
done; done

# 5) M1 표현 불일치 (RQ6) - 새 학습 0, 실행당 약 90초
python src/eval/run_tamper.py --track payload_4class_csicnorm --channels rgb

# 6) M2 인코딩 불변 채널 - npz 재빌드 → 재학습 → 회피 재평가
python src/imaging/build_image_dataset.py --track payload_4class_csicnorm \
    --channels rgb --rgb-encoders raw_byte,normalized_char_class,local_entropy

# 커밋 전 필수 (~3초, GPU 불필요)
python -m pytest tests/ -q
```

⚠ 재현성 관련 실사고 3건을 논문 부록에 함께 적습니다 — ① 지연 단발 측정이 speedup 을 9.30배로 부풀린 건(\$2.2), ② 산출물 tag 축 누락으로 서로 다른 실험이 덮여써진 건 3회(RGB 조합 · τ 축 · 캐스케이드 채널 축), ③ 기본 lr 이 한쪽 아키텍처에 맞춰져 있어 ViT

비교가 자동으로 불리해진 건. 세 건 모두 "같은 코드를 돌렸으니 공정하다"가 성립하지 않는 사례입니다.

14. 남은 결정 사항 (교수님 확인 필요)

#	내용	현재 권고
G6	RQ4b(USTC 흐름 트랙) 처리 방침 — F1 0.9999 포화로 판별력 없음	부록 강등 또는 host 단위 분할 재실험. 후자는 GPU 시간이 추가로 필요합니다.
—	FE(F1 Efficiency) 지표 채택 여부 — Tasdemir 2023 의 지표로 우리 캐스케이드를 평가하면 대조가 가장 깔끔해집니다. 우리는 τ 스왑이 있어 α 별 최적 τ 이동까지 그릴 수 있습니다(저쪽은 고정 운영점이라 불가).	채택 시 <code>metrics.py</code> 한 곳만 수정하면 됩니다.
—	H6-1 경계 밴드(단일 CNN p0.5 = 41.6%)의 5-fold CV 재판정 여부	캐스케이드 결과(69.2% · 77.9%)만으로 주장하면 불필요합니다.
—	캐스케이드 <code>norm</code> (B안) 판본 측정 여부 — 1.2차 정규화본 조립 + τ 재선택 필요	M2 vs B안을 캐스케이드 수준에서 대조하려면 필요합니다.
—	Phase 13 후보 — 배치 불변 이미지화(길이 정규화 리샘플링). \$6.2 의 진단이 옳다면 다음 수는 채널이 아니라 이쪽입니다.	줄업 일정에 따라 판단.

표 24. 논문 작성 착수 시점에 열려 있는 결정.

마무리 — 이 논문이 정직하게 말하는 것

이미지화는 더 정확하지 않습니다. 이 논문은 그 음성 결과에서 출발해, 이미지 표현의 유일한 실질 우위인 비용을 텍스트 모델의 정확도와 잇는 구조(캐스케이드)를 제안하고, 그 구조가 열어 준 새로운 공격면(지연)과 새로운 방어 대가(지연)를 실측했습니다. 시도한 확장 3안 중 2안은 부정 결과이며 그대로 실패합니다. 남은 하나(M2)는 이 프로젝트에서 가장 아팠던 음성 결과를 조건부로 뒤집었고, 그 실패 과정에서 "인코딩 회피의 위력은 값이 아니라 길이·배치에서 온다"는, 이미지화 탐지 계열 전반에 적용되는 진단을 얻었습니다.